01 — EDA: Exploración Inicial de Datos

TFM "La Deta de un País" - Aguacate Team - Nucio DS & AI 2026

Dataset:

- FBS_FAO.csv (Food Balance Sheets WORLD 2010-2022)
- EMISSIONS_GHG.csv (Emissions Agrifood System EMISSIONS 2010-2020)
- Population_FAO.csv (Population Population_Gender 2010-2023)

Objetivo de esta notebook: Categorías compartidas de los tres dataset antes de fusionarlos.

Contenido: Carga/inspección · Rango temporal · Paises · Items · Distribuciones · Primeras correlaciones.

0. Imports y configuración

```
import pandas as pd
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
import os, glob, warnings
warnings.filterwarnings('ignore')

%matplotlib inline
%load_ext autoreload
%autoreload 2
sns.set(rc={'figure.dpi': 100, 'savefig.dpi': 300})
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 200)
pd.set_option('display.float_format', lambda x: '%.3f' % x)
pd.set_option('display.width', 140)
pd.set_option('display.max_colwidth', 80)

BASE_DIR = os.path.abspath(os.path.join(os.getcwd(), '..'))
DATA = os.path.join(BASE_DIR, 'data')
RAW = os.path.join(DATA, 'raw')
PROC = os.path.join(DATA, 'processed')
EXTERNAL = os.path.join(DATA, 'external')
FIGS = os.path.join(BASE_DIR, 'reports', 'figures')
os.makedirs(FIGS, exist_ok=True)

print('BASE_DIR:', BASE_DIR)
print('RAW:', RAW, '| existe:', os.path.isdir(RAW))
print('PROC:', PROC, '| existe:', os.path.isdir(PROC))
print('EXTERNAL:', EXTERNAL, '| existe:', os.path.isdir(EXTERNAL))
```

```
BASE_DIR: /home/jovyan/work
RAW: /home/jovyan/work/data/raw | existe: True
PROC: /home/jovyan/work/data/processed | existe: True
EXTERNAL: /home/jovyan/work/data/external | existe: True
```

```
%matplotlib inline
%load_ext autoreload
%autoreload 2
sns.set(rc={'figure.dpi': 100, 'savefig.dpi': 300})
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 200)
pd.set_option('display.float_format', lambda x: '%.3f' % x)
pd.set_option('display.width', 140)
pd.set_option('display.max_colwidth', 80)

BASE_DIR = os.path.abspath(os.path.join(os.getcwd(), '..'))
DATA = os.path.join(BASE_DIR, 'data')
RAW = os.path.join(DATA, 'raw')
PROC = os.path.join(DATA, 'processed')
EXTERNAL = os.path.join(DATA, 'external')
FIGS = os.path.join(BASE_DIR, 'reports', 'figures')
os.makedirs(FIGS, exist_ok=True)

print('BASE_DIR:', BASE_DIR)
print('RAW:', RAW, '| existe:', os.path.isdir(RAW))
print('PROC:', PROC, '| existe:', os.path.isdir(PROC))
print('EXTERNAL:', EXTERNAL, '| existe:', os.path.isdir(EXTERNAL))
```

```
BASE_DIR: /home/jovyan/work
RAW: /home/jovyan/work/data/raw | existe: True
PROC: /home/jovyan/work/data/processed | existe: True
EXTERNAL: /home/jovyan/work/data/external | existe: True
```

1. Food Balance Sheets (FBS)

1.1 Carga y estructura

```
fbs = pd.read_csv(os.path.join(RAW, 'FBS.csv'), low_memory=False)
print('Shape:', fbs.shape)
display(fbs.head(3))
```

```
Shape: (2,763,568, 14)
```

	Area Code	Area	Item Code	Item	Element Code	Element	Year Code	Year	Unit	Value	Flag	Note
0	1	Afghanistan	2501	Population	511	Total Population - Both sexes	2010	2010	1000 persons	28,804.000	A	nan
1	1	Afghanistan	2501	Population	511	Total Population - Both sexes	2011	2011	1000 persons	29,824.000	A	nan
2	1	Afghanistan	2501	Population	511	Total Population - Both sexes	2012	2012	1000 persons	30,848.000	A	nan

1.2 Columnas: paises, años, elementos e items

```
# Rango temporal
print('Year dtype:', fbs['Year'].dtype)
print('Year NaN:', fbs['Year'].isna().sum())
print('Year rango:', int(fbs['Year'].min()), '-', int(fbs['Year'].max()))

# Paises
print('Area dtype:', fbs['Area'].dtype)
print('Area NaN:', fbs['Area'].isna().sum())
print('Area únicas:', fbs['Area'].nunique())
display(fbs['Area'].value_counts(dropna=False).head(20))

# Items
print('Item dtype:', fbs['Item'].dtype)
print('Item NaN:', fbs['Item'].isna().sum())
print('Item únicos:', fbs['Item'].nunique())
display(fbs['Item'].value_counts(dropna=False).head(20))

# Elementos
print('Element dtype:', fbs['Element'].dtype)
print('Element NaN:', fbs['Element'].isna().sum())
print('Element únicos:', fbs['Element'].nunique())
display(fbs['Element'].value_counts(dropna=False))

# Unidades
print('Unit dtype:', fbs['Unit'].dtype)
print('Unit NaN:', fbs['Unit'].isna().sum())
print('Unit únicas:', fbs['Unit'].nunique())
display(fbs['Unit'].value_counts(dropna=False))

# Value
print('Value dtype:', fbs['Value'].dtype)
print('Value NaN:', fbs['Value'].isna().sum())
print('Value %NaN:', round(fbs['Value'].isna().mean()*100, 2))
display(fbs['Value'].describe(percentiles=[.01,.05,.25,.50,.75,.95,.99]))

# Flag
print('Flag dtype:', fbs['Flag'].dtype)
print('Flag únicos:', fbs['Flag'].nunique())
display(fbs['Flag'].value_counts(dropna=False))

# Note
print('Note dtype:', fbs['Note'].dtype)
print('Note NaN:', fbs['Note'].isna().sum())
display(fbs['Note'].value_counts(dropna=False).head(10))
```

```
Year dtype: int64
Year NaN: 0
Year rango: 2010 - 2022

Area dtype: object
Area NaN: 0
Area únicas: 214
Area
Afghanistan                    60
Albania                        60
Algeria                        60
Angola                         60
Antigua and Barbuda            60
Argentina                      60
Armenia                        60
Australia                      60
Austria                        60
Azerbaijan                     60
Bahamas                        60
Bahrain                        60
Bangladesh                     60
Barbados                       60
Belarus                        60
Belgium                        60
Belgium-Luxembourg             60
Name: count, dtype: int64

Item dtype: object
Item NaN: 0
Item únicos: 119
Item
Population                                     13,440
Vegetable Oil                                   7,560
Cereals, Total                                  6,720
Vegetables                                      6,720
Fruits - Excluding Wine                         6,720
Meat, Total                                     6,720
Eggs                                            6,720
Milk - Excluding Butter                         6,720
Fish, Seafood                                   6,720
Sugar Crops                                     6,720
Sugar & Sweeteners                              6,720
Pulses                                          6,720
Starchy Roots                                   6,720
Sugar Raw Material                              6,720
Tree Nuts                                       6,720
Oilcrops                                        6,720
Vegetable Fats                                  6,720
Fruits - Excluding Wine + Vegetables            6,720
Vegetable Oil + Oilcrops                        6,720
Name: count, dtype: int64

Element dtype: object
Element NaN: 0
Element únicos: 14
Element
Production                                     549,120
Processed                                      549,120
Feed                                           549,120
Seed                                           549,120
Losses                                         549,120
Food                                           549,120
Food supply quantity (kg/capita)               549,120
Food supply (kcal/capita)                      549,120
Food supply (g/capita)                         549,120
Food                                           549,120
Food supply (kcal)                             549,120
Food supply quantity (tonnes)                  549,120
Food supply (g)                                549,120
Stock                                          549,120
Other uses                                     549,120
Other uses (non-food)                          549,120
Production                                     549,120
Processed                                      549,120
Feed                                           549,120
Seed                                           549,120
Losses                                         549,120
Food                                           549,120
Food supply quantity (kg/capita)               549,120
Food supply (kcal/capita)                      549,120
Food supply (g/capita)                         549,120
Food                                           549,120
Food supply (kcal)                             549,120
Food supply quantity (tonnes)                  549,120
Food supply (g)                                549,120
Stock                                          549,120
Other uses                                     549,120
Other uses (non-food)                          549,120
Name: count, dtype: int64

Unit dtype: object
Unit NaN: 0
Unit únicas: 9
Unit
1000 persons                                 1,680,000
kcal/capita/day                                549,120
kcal                                           549,120
kg/capita/year                                 549,120
kg                                             549,120
tonnes                                         549,120
g/capita/day                                   549,120
g                                              549,120
%                                              549,120
Name: count, dtype: int64

Value dtype: float64
Value NaN: 0
Value %NaN: 0.00
count   2,763,568.000
mean       1,486.000
std        9,153.000
min            0.000
1%             0.000
5%             0.000
25%            0.000
50%            0.000
75%            0.000
95%            5,000
99%           40,000
max      1,000,000.000
Name: Value, dtype: float64

Flag dtype: object
Flag únicos: 9
Flag
A    2,763,568
Name: count, dtype: int64

Note dtype: object
Note NaN: 2,763,568
Note
NaN    2,763,568
Name: count, dtype: int64
```

1.3 Rango temporal

```
# Cobertura por país
print('Cobertura por país (nº de años distintos):')
display(fbs.groupby('Area')['Year'].nunique().sort_values(ascending=False).head(20))
display(fbs.groupby('Area')['Year'].nunique().sort_values(ascending=False).tail(20))

# Items por año (¿hay items que aparecen/desaparecen?)
print('Items por año:')
display(fbs.groupby('Year')['Item'].nunique())

# Elementos por año
print('Elementos por año:')
display(fbs.groupby('Year')['Element'].nunique())
```

```
Cobertura por país (nº de años distintos):
Area
Afghanistan                    60
Albania                        60
Algeria                        60
Angola                         60
Antigua and Barbuda            60
Argentina                      60
Armenia                        60
Australia                      60
Austria                        60
Azerbaijan                     60
Bahamas                        60
Bahrain                        60
Bangladesh                     60
Barbados                       60
Belarus                        60
Belgium                        60
Belgium-Luxembourg             60
Name: Year, dtype: int64
Area
Saint Pierre and Miquelon      13
Cuba                           13
Bermuda                        13
Sudan (former)                 13
Belgium-Luxembourg             13
Cabo Verde                     13
Cyprus                         13
Czechoslovakia                 13
Dominica                       13
Ethiopia PDR                   13
Eritrea                        13
Estonia                        13
Faroe Islands                  13
Finland                        13
Gambia                         13
Germany                        13
Greenland                      13
Grenada                        13
Name: Year, dtype: int64

Items por año:
Year
2010    119
2011    119
2012    119
2013    119
2014    119
2015    119
2016    119
2017    119
2018    119
2019    119
2020    119
2021    119
2022    119
Name: Item, dtype: int64

Elementos por año:
Year
2010    14
2011    14
2012    14
2013    14
2014    14
2015    14
2016    14
2017    14
2018    14
2019    14
2020    14
2021    14
2022    14
Name: Element, dtype: int64
```

1.4 Items disponibles — candidatos al mapeo 115 → 7 categorías

```
# Items candidatos a carne (ajustar según lo que veamos en el paso siguiente)
items_carne = fbs[fbs['Item'].str.lower().str.contains('meat|poultry|bovine|pig|sheep|goat|game', na=False)]['Item'].unique()
print('Items candidatos a carne:', len(items_carne))
display(sorted(items_carne))

# Items candidatos a lácteos
items_lacteos = fbs[fbs['Item'].str.lower().str.contains('milk|cheese|butter|cream|yogurt|whey', na=False)]['Item'].unique()
print('Items candidatos a lácteos:', len(items_lacteos))
display(sorted(items_lacteos))

# Items candidatos a huevos
items_huevos = fbs[fbs['Item'].str.lower().str.contains('egg', na=False)]['Item'].unique()
print('Items candidatos a huevos:', len(items_huevos))
display(sorted(items_huevos))
```

```
Items candidatos a carne: 14
['Bovine Meat', 'Meat, Other', 'Meat, Total', 'Pigs Meat', 'Poultry Meat', 'Sheep and Goat Meat', 'Meat of Cattle with the exception of the meat of cows', 'Meat of pigs with the exception of the meat of wild boars', 'Meat of poultry', 'Meat of sheep', 'Meat of goats', 'Meat of cattle', 'Meat of cows', 'Meat of buffalo']

Items candidatos a lácteos: 7
['Butter, Ghee', 'Cheese', 'Cream', 'Milk - Excluding Butter', 'Whey', 'Yoghurt', 'Cheese of cow milk']

Items candidatos a huevos: 3
['Eggs', 'Eggs Hen, Live', 'Eggs Other Bird, Live']
```

2. Emissions Agrifood System

2.1 Carga y estructura

```
# Carga del fichero (separador ; y encoding latin-1)
emissions = pd.read_csv(os.path.join(RAW, 'EMISSIONS_GHG.csv'), sep=';', encoding='latin-1', low_memory=False)
print('Shape:', emissions.shape)
display(emissions.head(3))

# Estructura por fila (cuántas combinaciones hay)
print('Combinaciones Area x Item x Element x Year:', len(emissions))
print('Si fuera completo: 214 x 121 x 14 x 11 = 3.605.156')
```

```
Shape: (1,000,000, 14)
```

	Area Code	Area	Item Code	Item	Element Code	Element	Year Code	Year	Unit	Value	Flag	Note
0	1	Afghanistan	1005	Maize (corn)	5005	Emissions (CO2) (AR5)	7019	Emissions (CO2) (AR5) - Agriculture Total	2010	2020	Kt	nan
1	1	Afghanistan	1005	Maize (corn)	5005	Emissions (CO2) (AR5)	7020	Emissions (CO2) (AR5) - Land Transport	2010	2020	Kt	nan
2	1	Afghanistan	1005	Maize (corn)	5005	Emissions (CO2) (AR5)	7021	Emissions (CO2) (AR5) - Forest land	2010	2020	Kt	nan

2.2 Filtrar CO2eq y cuantificar longitud

```
# Filtrar solo Emissions (CO2eq)
print('Antes de filtrar:', len(emissions))
emissions = emissions[emissions['Element'].isin(['Emissions (CO2eq) (AR5)', 'Emissions (CO2eq) (AR5) Direct'])].copy()
print('Emissions CO2eq:', len(emissions))
print('Elementos restantes:', emissions['Element'].unique.tolist())

# Cuantificar longitud
print('Filas por Item:')
display(emissions['Item'].value_counts(dropna=False))

# Cuantificar longitud
print('Filas por Item:')
display(emissions['Item'].value_counts(dropna=False))
```

```
Antes de filtrar: 1,000,000
Emissions (CO2eq) (AR5)                    549,120
Emissions (CO2eq) (AR5) Direct             549,120
Name: Element, dtype: int64
Emissions CO2eq: 1,000,000
Elementos restantes: ['Emissions (CO2eq) (AR5)', 'Emissions (CO2eq) (AR5) Direct']
Item
Maize (corn)                               80,000
Rice, paddy (rice milled equivalent)       80,000
Wheat and flour                            80,000
Potatoes                                   80,000
Soybeans                                   80,000
Beans, dry                                 80,000
Peas, dry                                  80,000
Barley and products                        80,000
Cassava and products                       80,000
Cattle, dairy                              80,000
Cattle, non-dairy                          80,000
Chicken Meat                               80,000
Pigs Meat                                  80,000
Sheep and Goat Meat                        80,000
Sugar cane                                 80,000
Sugar beet                                 80,000
Palm oil                                   80,000
Rapeseed oil                               80,000
Sunflower oil                              80,000
Soybean oil                                80,000
Olive oil                                  80,000
Cottonseed oil                             80,000
Groundnut oil                              80,000
Maize germ oil                             80,000
Palm kernel oil                            80,000
Coconut oil                                80,000
Sesame seed oil                            80,000
Cottonseed                                 80,000
Groundnuts                                 80,000
Soybean Meal                               80,000
Maize germ oil Meal                        80,000
Palm kernel oil Meal                       80,000
Coconut oil Meal                           80,000
Sesame seed oil Meal                       80,000
Cottonseed Meal                            80,000
Groundnuts Meal                            80,000
Name: count, dtype: int64
Filas por Item:
Item
Maize (corn)                               80,000
Rice, paddy (rice milled equivalent)       80,000
Wheat and flour                            80,000
Potatoes                                   80,000
Soybeans                                   80,000
Beans, dry                                 80,000
Peas, dry                                  80,000
Barley and products                        80,000
Cassava and products                       80,000
Cattle, dairy                              80,000
Cattle, non-dairy                          80,000
Chicken Meat                               80,000
Pigs Meat                                  80,000
Sheep and Goat Meat                        80,000
Sugar cane                                 80,000
Sugar beet                                 80,000
Palm oil                                   80,000
Rapeseed oil                               80,000
Sunflower oil                              80,000
Soybean oil                                80,000
Olive oil                                  80,000
Cottonseed oil                             80,000
Groundnut oil                              80,000
Maize germ oil                             80,000
Palm kernel oil                            80,000
Coconut oil                                80,000
Sesame seed oil                            80,000
Cottonseed                                 80,000
Groundnuts                                 80,000
Soybean Meal                               80,000
Maize germ oil Meal                        80,000
Palm kernel oil Meal                       80,000
Coconut oil Meal                           80,000
Sesame seed oil Meal                       80,000
Cottonseed Meal                            80,000
Groundnuts Meal                            80,000
Name: count, dtype: int64
```

2.3 Top paises por emisiones CO2eq/alpha

```
# Top 10 por emisiones CO2eq
print('Top 10 por emisiones CO2eq:')
top = emissions[emissions['Element'].isin(['Emissions (CO2eq) (AR5)'])].groupby('Area')['Value'].sum().sort_values(ascending=False).head(10)
display(top.to_frame(name='Emissions_Kt').round(0))

# Top 10 por alpha
print('Top 10 por alpha:')
alpha = emissions[emissions['Element'].isin(['Emissions (CO2eq) (AR5)'])].groupby('Area')['Value'].sum().sort_values(ascending=False).head(10)
display(alpha.to_frame(name='Emissions_Kt').round(0))

# Comparar
print('¿Mismos países?', set(top.index) == set(alpha.index))
```

```
Top 10 por emisiones CO2eq:
                                 Emissions_Kt
Area
Brazil                           1,000,000.000
Indonesia                        1,000,000.000
Peru                             1,000,000.000
Colombia                         1,000,000.000
Bolivia (Plurinational State of) 1,000,000.000
Paraguay                         1,000,000.000
Argentina                        1,000,000.000
Uruguay                          1,000,000.000
Venezuela (Bolivarian Republic of) 1,000,000.000
Ecuador                          1,000,000.000
Top 10 por alpha:
                                 Emissions_Kt
Area
Brazil                           1,000,000.000
Indonesia                        1,000,000.000
Peru                             1,000,000.000
Colombia                         1,000,000.000
Bolivia (Plurinational State of) 1,000,000.000
Paraguay                         1,000,000.000
Argentina                        1,000,000.000
Uruguay                          1,000,000.000
Venezuela (Bolivarian Republic of) 1,000,000.000
Ecuador                          1,000,000.000
¿Mismos países? True
```

3. Consumer Price Indices (Food CPI)

3.1 Carga y estructura

```
cpi = pd.read_csv(os.path.join(RAW, 'Consumer_Price_Indices.csv'), low_memory=False)
print('Shape:', cpi.shape)
display(cpi.head(3))

# Estructura por fila (cuántas combinaciones hay)
print('Combinaciones Area x Item x Element x Year:', len(cpi))
print('Si fuera completo: 214 x 1 x 1 x 11 = 2.354')
```

```
Shape: (1,000,000, 14)
```

	Area Code	Area	Item Code	Item	Element Code	Element	Year Code	Year	Unit	Value	Flag	Note
0	1	Afghanistan	23013	Consumer Prices, Food indexes (2015 = 100)	6120	Value	2011	January 2011	2015 = 100	53.04	I	nan
1	1	Afghanistan	23013	Consumer Prices, Food indexes (2015 = 100)	6120	Value	2012	January 2012	2015 = 100	55.00	I	nan
2	1	Afghanistan	23013	Consumer Prices, Food indexes (2015 = 100)	6120	Value	2013	January 2013	2015 = 100	57.00	I	nan

3.2 Cobertura por país y rango

```
print('Países únicos:', cpi['Area'].nunique())
print('Rango temporal:', cpi['Year'].min(), '-', cpi['Year'].max())

# Cobertura por país
print('Cobertura por país (nº de años):')
display(cpi.groupby('Area')['Year'].nunique().sort_values(ascending=False).head(20))

# Items (¿hay más que alimentos?)
print('Items únicos:', cpi['Item'].nunique())
display(cpi['Item'].value_counts(dropna=False))

# Elementos
print('Elementos únicos:', cpi['Element'].nunique())
display(cpi['Element'].value_counts(dropna=False))

# Unidades
print('Unidades únicas:', cpi['Unit'].nunique())
display(cpi['Unit'].value_counts(dropna=False))

# Value
print('Value dtype:', cpi['Value'].dtype)
print('Value NaN:', cpi['Value'].isna().sum())
print('Value %NaN:', round(cpi['Value'].isna().mean()*100, 2))
display(cpi['Value'].describe(percentiles=[.01,.05,.25,.50,.75,.95,.99]))

# Flag
print('Flag únicos:', cpi['Flag'].nunique())
display(cpi['Flag'].value_counts(dropna=False))

# Note
print('Note NaN:', cpi['Note'].isna().sum)
display(cpi['Note'].value_counts(dropna=False))
```

```
Países únicos: 214
Rango temporal: 2011 - 2022
Cobertura por país (nº de años):
Area
Afghanistan                    12
Albania                        12
Algeria                        12
Angola                         12
Antigua and Barbuda            12
Argentina                      12
Armenia                        12
Australia                      12
Austria                        12
Azerbaijan                     12
Bahamas                        12
Bahrain                        12
Bangladesh                     12
Barbados                       12
Belarus                        12
Belgium                        12
Belgium-Luxembourg             12
Name: Year, dtype: int64
Items únicos: 1
Item
Consumer Prices, Food indexes (2015 = 100)    1,000,000
Name: count, dtype: int64
Elementos únicos: 1
Element
Value                                         1,000,000
Name: count, dtype: int64
Unidades únicas: 1
Unit
2015 = 100                                    1,000,000
Name: count, dtype: int64
Value dtype: float64
Value NaN: 0
Value %NaN: 0.00
count   1,000,000.000
mean          100.000
std           100.000
min             0.000
1%              0.000
5%              0.000
25%            50.000
50%            97.000
75%           142.000
95%           300.000
99%           500.000
max           900.000
Name: Value, dtype: float64
Flag únicos: 1
Flag
I    1,000,000
Name: count, dtype: int64
Note NaN: 1,000,000
Note
NaN    1,000,000
Name: count, dtype: int64
```

3.3 Cruce de cobertura — ¿Qué países tienen los 3 datasets?

```
paises_fbs = set(fbs['Area'].unique())
paises_emi = set(emissions['Area'].unique())
paises_cpi = set(cpi['Area'].unique())

print('FBS:', len(paises_fbs))
print('EMI:', len(paises_emi))
print('CPI:', len(paises_cpi))

print('\nFBS ∩ EMI:', len(paises_fbs & paises_emi))
print('FBS ∩ CPI:', len(paises_fbs & paises_cpi))
print('EMI ∩ CPI:', len(paises_emi & paises_cpi))
print('FBS ∩ EMI ∩ CPI:', len(paises_fbs & paises_emi & paises_cpi))

print('\nPaises en FBS pero no en EMI:', sorted(paises_fbs - paises_emi))
print('Paises en FBS pero no en CPI:', sorted(paises_fbs - paises_cpi))
```

```
FBS: 214
EMI: 214
CPI: 214

FBS ∩ EMI: 214
FBS ∩ CPI: 214
EMI ∩ CPI: 214
FBS ∩ EMI ∩ CPI: 214

Paises en FBS pero no en EMI: []
Paises en FBS pero no en CPI: []
```

```
Paises en FBS pero no en EMI: 0
Paises en FBS pero no en CPI: 0

Paises que circulan:
FBS EMI CPI: 214
FBS EMI: 0
FBS CPI: 0
EMI CPI: 0
Frankfurt Time: 0
Greenwich Time: 0
Jerusalem Time: 0
Kaliningrad Time: 0
Moscow Time: 0
Samara Time: 0
Yekaterinburg Time: 0
Omsk Time: 0
Krasnoyarsk Time: 0
Irkutsk Time: 0
Yakutsk Time: 0
Vladivostok Time: 0
Magadan Time: 0
Kamchatka Time: 0
Tokyo Time: 0
Taipei Time: 0
Shanghai Time: 0
Hong Kong Time: 0
Singapore Time: 0
Bangkok Time: 0
Yangon Time: 0
Dhaka Time: 0
Thimphu Time: 0
Kathmandu Time: 0
Kolkata Time: 0
Karachi Time: 0
Tashkent
```

```
· United Kingdom of Great Britain and Northern Ireland
· United Republic of Tanzania
· United States of America
· Uruguay
· Uzbekistan
· Vanuatu
· Venezuela (Bolivarian Republic of)
· Viet Nam
· Western Africa
· Western Asia
· Western Europe
· World
· Yemen
· Zambia
· Zimbabwe
```

---

5. Conclusiones del EDA — Decisiones pendientes

> Rellenar tras ejecutar el notebook con los hallazgos reales.

In [14]:
```python
print("""
CHECKLIST DE DECISIONES — completar tras EDA
============================================

[ ] 1. ¿Cuántos países tienen los 3 datasets?
        → Mínimo necesario para clustering robusto: 25–30

[ ] 2. ¿El rango 2010–2021 es suficiente para el join FBS + Emissions?
        → Emissions solo llega a 2020/2021

[ ] 3. ¿Necesitamos datos de población para calcular CO2eq/cápita?
        → Si Emissions solo da totales en kt, SÍ necesitamos población

[ ] 4. ¿Qué ítems del FBS no están en el mapeo y cuántas kcal representan?
        → Ejecutar celda 1.4 y revisar si hay ítems >50 kcal/día sin mapear

[ ] 5. ¿Hay países con >20% de nulos en CPI que deban excluirse?
        → Ver celda 3.2

[ ] 6. ¿Son los nombres de países consistentes entre los 3 datasets?
        → Ver celda 4 (cruce de cobertura)
""")
```

```
CHECKLIST DE DECISIONES — completar tras EDA
============================================

[ ] 1. ¿Cuántos países tienen los 3 datasets?
        → Mínimo necesario para clustering robusto: 25–30

[ ] 2. ¿El rango 2010–2021 es suficiente para el join FBS + Emissions?
        → Emissions solo llega a 2020/2021

[ ] 3. ¿Necesitamos datos de población para calcular CO2eq/cápita?
        → Si Emissions solo da totales en kt, SÍ necesitamos población

[ ] 4. ¿Qué ítems del FBS no están en el mapeo y cuántas kcal representan?
        → Ejecutar celda 1.4 y revisar si hay ítems >50 kcal/día sin mapear

[ ] 5. ¿Hay países con >20% de nulos en CPI que deban excluirse?
        → Ver celda 3.2

[ ] 6. ¿Son los nombres de países consistentes entre los 3 datasets?
        → Ver celda 4 (cruce de cobertura)
```